



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.I. Edgar Tista García

Asignatura: Estructura de Datos y Algoritmos I

Grupo: 01

No de Práctica(s): 04

Integrante(s): Jiménez Olivo Evelin

*No. de lista o
brigada:* 17

Semestre: 2024-2

Fecha de entrega: 6 de marzo del 2024

Observaciones:

CALIFICACIÓN: _____

Práctica 4. Almacenamiento en tiempo de ejecución.

Objetivo:

Utilizar funciones en lenguaje C que permiten reservar y almacenar información de manera dinámica (en tiempo de ejecución).

Objetivo de clase:

Comprender el uso básico de apuntadores aplicado al almacenamiento en tiempo de ejecución, así como las funciones esenciales para el trabajo de los mismos

Código (malloc)

¿Qué hace el programa?

Es un programa en el cual se solicita ingresar la cantidad de elementos que desea tener en un conjunto, se reserva memoria dinámica para almacenar esos elementos, se muestra la memoria reservada, aunque el conjunto tendrá valores indeterminados, ya que no se ha inicializado las localidades de memoria, se libera la memoria reservada y luego finaliza.

Para esto primero se declaran las librerías “stdio.h” y “stdlib.h”, esta última ya que ahí está definida la función malloc, se declaran las variables, un apuntador a un arreglo de tipo entero para almacenar los elementos del conjunto, un entero “num” para almacenar la cantidad de elementos que el usuario ingresará, y un contador cont para el ciclo for. Se pide que se ingrese la cantidad de elementos del conjunto y el valor lo almacena en la variable “num”. Luego se utiliza la función malloc() para reservar memoria dinámica, la cantidad de memoria reservada es el numero “num” que ingrese el usuario.

```
0 | scanf("%d", &num);  
7 | arreglo = (int *)malloc (num * sizeof(int));  
- |
```

Posteriormente se verifica si la reserva de memoria fue exitosa con un if, si fue exitosa se muestra un mensaje indicando que se ha reservado el vector. Con el ciclo for se itera cada elemento del arreglo e imprime su valor (aunque son valores indeterminados).

```
for (cont=0 ; cont<num ; cont++)
    printf("\t%d",*(arreglo+cont));
```

Finalmente se libera la memoria reservada utilizando la función “free()”.

```
free(arreglo);
```

¿Cuál es la salida que se muestra en pantalla?

La salida del programa es un mensaje el cual se le pide al usuario que ingresa la cantidad de elementos que tendrá el conjunto, ya que el usuario lo ingresa, se muestra el vector reservado, sin embargo, se muestran valores “extraños” ya que la no se inicializó las localidades de memoria.

```
C:\Users\eve18\OneDrive\Doc >
¿Cuántos elementos tiene el conjunto?
5
Vector reservado:
[ 12085280 0 12058960 0 942761334 ]
Se libera el espacio reservado.

-----
Process exited after 2.107 seconds with return value 0
Presione una tecla para continuar . . .
```

Finalmente se muestra el mensaje de que se liberó el espacio reservado.

Si el programa tiene errores, especificar cuáles son y corregirlos

Un “error” es que no se inicializan las localidades de memoria por lo que al imprimir se muestran “valores extraños” por lo que inicializamos las localidades asignando al elemento del arreglo, lo que hay en el valor contador + 1.

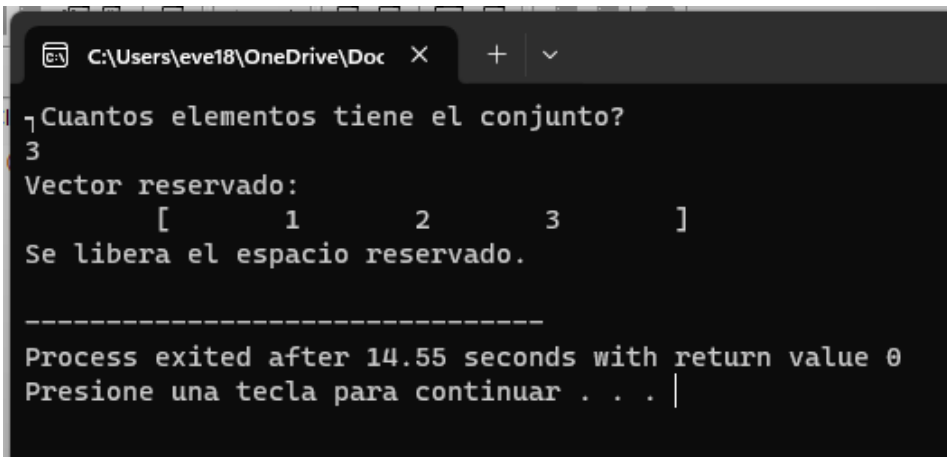
```
arreglo[cont] = cont + 1;
```

Error

```
C:\Users\eve18\OneDrive\Doc >
¿Cuántos elementos tiene el conjunto?
3
Vector reservado:
[ 11036704 0 11010384 ]
Se libera el espacio reservado.

-----
Process exited after 15.72 seconds with return value 0
Presione una tecla para continuar . . .
```

Corregido



```
C:\Users\eve18\OneDrive\Doc > C:\Users\eve18\OneDrive\Doc >
1 Cuantos elementos tiene el conjunto?
2 3
3 Vector reservado:
4 [ 1 2 3 ]
5 Se libera el espacio reservado.
6
7 -----
8 Process exited after 14.55 seconds with return value 0
9 Presione una tecla para continuar . . . |
```

Comentario acerca de la relación del ejemplo con el concepto teórico

La función `malloc()` se utiliza en este programa para reservar memoria dinámica para un arreglo de enteros.

Cuando el usuario ingresa la cantidad de elementos que desea tener en el conjunto, el programa utiliza ese número para determinar cuánta memoria necesita reservar para el arreglo, igual se utiliza la función “`sizeof`” para saber la cantidad de localidad de memoria que se requiere. La función `malloc()` se encarga de solicitar esa cantidad específica de memoria del sistema operativo.

```
arreglo = (int *)malloc (num * sizeof(int));
```

Por ejemplo, si el usuario ingresa 3 como la cantidad de elementos del conjunto, el programa calculará el tamaño necesario para almacenar 3 enteros contiguos en la memoria. Luego, `malloc()` asigna dinámicamente esa cantidad de memoria y devuelve un apuntador a la dirección de memoria reservada.

Finalmente para liberar la memoria se utiliza la función `free()`.

Código (calloc)

¿Qué hace el programa?

Es un programa en el cual se solicita ingresar la cantidad de elementos que desea tener en un conjunto, se reserva memoria dinámica para almacenar esos elementos utilizando la función `calloc()`, se muestra la memoria reservada, inicializa todos los elementos del arreglo a cero, libera la memoria reservada y se finaliza.

Para esto primero se declaran las librerías “stdio.h” y “stdlib.h”, esta última ya que ahí está definida la función calloc (), se declaran las variables, un apuntador a un arreglo de tipo entero para almacenar los elementos del conjunto, un entero “num” para almacenar la cantidad de elementos que el usuario ingresará, y un contador cont para el ciclo for. Se pide que se ingrese la cantidad de elementos del conjunto y el valor lo almacena en la variable “num”. Luego se utiliza la función calloc() para reservar memoria dinámica para el arreglo. En esta función se asignan como parámetros el número de localidades a reservar que es la variable “num” y el tamaño de esas localidades con “sizeof”; se establecen como ceros los valores iniciales de las localidades de memoria que reserva

```
9 arreglo = (int *)calloc (num, sizeof(int));
```

Posteriormente se verifica si la reserva de memoria fue exitosa con un if, si fue exitosa se muestra un mensaje indicando que se ha reservado el vector. Con el ciclo for se itera cada elemento del arreglo e imprime su valor (las cuales son ceros).

```

13
14
15
16

```

```

for (cont=0 ; cont<num ; cont++)
{
    printf("\t%d",*(arreglo+cont));
}

```

Finalmente se libera la memoria reservada utilizando la función “free()”.

¿Cuál es la salida que se muestra en pantalla?

La salida del programa es un mensaje el cual se le pide al usuario que ingresa la cantidad de elementos que tendrá el conjunto, ya que el usuario lo ingresa, se muestra el vector reservado, sin embargo, se muestran todos los valores como ceros, ya que no se inicializan las localidades de memoria por lo que al utilizar la función calloc() se establecen como ceros los valores iniciales de las localidades de memoria

```
C:\Users\eve18\OneDrive\Doc X + v
7 Cuantos elementos tiene el conjunto?
3
Vector reservado:
      [      0      0      0      ]
Se libera el espacio reservado.

-----
Process exited after 25.3 seconds with return value 0
Presione una tecla para continuar . . . |
```

Finalmente se muestra el mensaje de que se liberó el espacio reservado.

Comentario acerca de la relación del ejemplo con el concepto teórico

La función `calloc()` se utiliza en este programa para reservar memoria dinámica para un arreglo de enteros.

Cuando el usuario ingresa la cantidad de elementos que desea tener en el conjunto, el programa utiliza ese número para determinar cuánta memoria necesita reservar para el arreglo, igual se utiliza la función “`sizeof`” para saber la cantidad de localidad de memoria que se requiere, estos dos datos los usa como parámetros la función `calloc()`.

```
arreglo = (int *)calloc (num * sizeof(int));
```

La función `calloc()` igual se utiliza para inicializa todos los elementos de la localidad de memoria en cero.

Código (realloc)

¿Qué hace el programa?

Lo que hace el programa es solicitar ingresar la cantidad de elementos que se desea tener en el conjunto, luego se solicita que se ingrese cada elemento del conjunto uno por uno. Después de que todos los elementos hayan sido ingresados, el programa muestra el conjunto original, luego duplica el tamaño del conjunto y se solicita que se ingrese los elementos adicionales para el conjunto ampliado. Finalmente, muestra el conjunto ampliado y libera la memoria asignada dinámicamente antes de finalizar.

Para esto primero se declaran las librerías “stdio.h” y “stdlib.h”, se declaran las variables, un apuntador a un arreglo de tipo entero para almacenar los elementos del conjunto original, y otro apuntador a un arreglo2 de tipo entero para almacenar los elementos del conjunto ampliado, un entero “num” para almacenar la cantidad de elementos que el usuario ingresará, y un contador cont para el ciclo for. Se pide que se ingrese la cantidad de elementos del conjunto y el valor lo almacena en la variable “num”. Luego se utiliza la función malloc() para reservar memoria dinámica para el arreglo original, con la cantidad de elementos especificada por el usuario.

```
9 | arreglo = (int *)malloc (num * sizeof(int));
```

Posteriormente se verifica si la reserva de memoria fue exitosa con un if, si fue exitosa se solicita que se ingrese cada elemento del conjunto uno utilizando un ciclo for. Después de que todos los elementos se ingresaron, se muestra el conjunto original, es decir el usuario inicializa cada localidad de memoria.

```
for (cont=0 ; cont < num ; cont++)  
{  
    printf("Inserte el elemento %d del conjunto.\n",cont+1);  
    scanf("%d",(arreglo+cont));  
}
```

Después se duplica el tamaño del conjunto multiplicando la variable “num” por 2 y se utiliza la función realloc() para redimensionar el arreglo original al nuevo tamaño, para eso se utilizan como parámetros el apuntador que se va a redimensionar y el nuevo valor de num por el tamaño de la variable.

```
24 | num *= 2;  
25 | arreglo2 = (int *)realloc (arreglo,num*sizeof(int));
```

Se verifica si la redimensión de memoria fue exitosa con otro if, si fue exitosa se solicita que se ingresen los elementos adicionales para el conjunto ampliado

utilizando un ciclo for. Posteriormente de que todos los elementos adicionales se ingresaron, se muestra el conjunto ampliado.

```
arreglo = arreglo2;
for (; cont < num ; cont++)
{
    printf("Inserte el elemento %d del conjunto.\n",cont+1);
    scanf("%d",&arreglo2[cont]);
}
```

Finalmente se libera la memoria reservada utilizando la función "free()".

¿Cuál es la salida que se muestra en pantalla?

La salida es que se piden ingresar cuantos elementos va a tener el conjunto, posteriormente se piden ingresar cada elemento del conjunto, luego se imprime el vector insertado.

Luego se indica que el tamaño se aumento al doble y se piden ingresar los elementos adicionales del conjunto, finalmente se imprime el vector final.

```
¿Cuántos elementos tiene el conjunto?
3
Inserte el elemento 1 del conjunto.
3
Inserte el elemento 2 del conjunto.
4
Inserte el elemento 3 del conjunto.
5
Vector insertado:
      [      3      4      5      ]
Aumentando el tamaño del conjunto al doble.
Inserte el elemento 4 del conjunto.
6
Inserte el elemento 5 del conjunto.

7
Inserte el elemento 6 del conjunto.
8
Vector insertado:
      [      3      4      5      6      7      8      ]

-----
Process exited after 232.3 seconds with return value 0
Presione una tecla para continuar . . .
```


Comentario acerca de la relación del ejemplo con el concepto teórico

La función malloc() se utiliza en este programa para reservar memoria dinámica para un arreglo de enteros.

Cuando el usuario ingresa la cantidad de elementos que desea tener en el conjunto, el programa utiliza ese número para determinar cuánta memoria necesita reservar para el arreglo, igual se utiliza la función "sizeof" para saber la cantidad de localidad de memoria que se requiere. La función malloc() se encarga de solicitar esa cantidad específica de memoria del sistema operativo.

```
arreglo = (int *)malloc (num * sizeof(int));
```

Después se utiliza la función realloc() para redimensionar el espacio asignado del arreglo al doble.

```
25 |         arreglo2 = (int *)realloc (arreglo,num*sizeof(int));
```

Finalmente para liberar la memoria se utiliza la función free().

Ejercicio 1

Compila y ejecuta el código “ejercicio1” posteriormente responde las preguntas.

a.- Captura la pantalla con la ejecución del programa y explica de manera detallada, los resultados que se observan.

```
direccion arreglo[0]=-1163348160  valor arreglo[0]=35
direccion arreglo[1]=-1163348156  valor arreglo[1]=40
direccion arreglo[2]=-1163348152  valor arreglo[2]=45
direccion arreglo[3]=-1163348148  valor arreglo[3]=50
direccion arreglo[4]=-1163348144  valor arreglo[4]=55
direccion arreglo[5]=-1163348140  valor arreglo[5]=32759
direccion arreglo[6]=-1163348136  valor arreglo[6]=324206622
direccion arreglo[7]=-1163348132  valor arreglo[7]=570233363
direccion arreglo[8]=-1163348128  valor arreglo[8]=-1163347472
direccion arreglo[9]=-1163348124  valor arreglo[9]=32759

direccion=37486624  *valor=0
direccion=37486628  *valor=0
direccion=37486632  *valor=0
direccion=37486636  *valor=0
direccion=37486640  *valor=0
direccion=37486644  *valor=0
direccion=37486648  *valor=0
direccion=37486652  *valor=0
direccion=37486656  *valor=0
direccion=37486660  *valor=0
```

Primero se imprimen las direcciones de memoria de todos los elementos del arreglo estático, luego se imprime su valor de los elementos del arreglo, como solo en el arreglo se inicializan 5 valores del arreglo y el for itera 10 valores, los otros 5 índices tienen valores y direcciones “extraños”, es decir no definidos ya que no están inicializados.

Los 5 elementos inicializados

```
arreglo[]={35,40,45,50,55};
```

```
valor arreglo[0]=35
valor arreglo[1]=40
valor arreglo[2]=45
valor arreglo[3]=50
valor arreglo[4]=55
```

Los otros 5 elementos sin valor inicializado

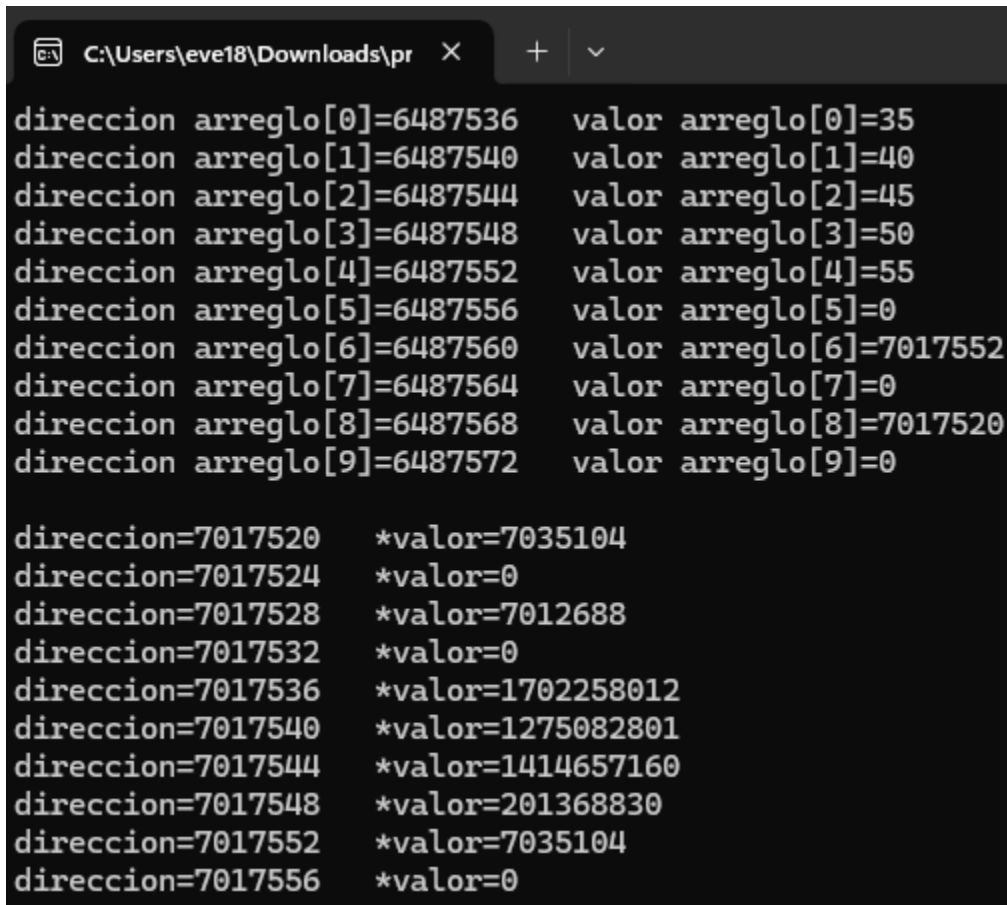
```
11 | for(cont=0;cont<10;cont++)
```

```
valor arreglo[5]=0
valor arreglo[6]=7017552
valor arreglo[7]=0
valor arreglo[8]=7017520
valor arreglo[9]=0
```

Luego utiliza otro ciclo for para imprimir las direcciones y el contenido del apuntador “*ptr” de memoria dinámica que se reserva con malloc().

```
7 | int *ptr = malloc(10);
```

Como no se inicializa imprime direcciones no inicializadas. Imprime valor de 0 porque en esas localidades de memoria que tomó malloc() está vacío, sin embargo puede suceder que se imprima basura u otros datos, esto depende de la computadora, por ejemplo en mi computadora se imprime lo siguiente.



```
direccion arreglo[0]=6487536 valor arreglo[0]=35
direccion arreglo[1]=6487540 valor arreglo[1]=40
direccion arreglo[2]=6487544 valor arreglo[2]=45
direccion arreglo[3]=6487548 valor arreglo[3]=50
direccion arreglo[4]=6487552 valor arreglo[4]=55
direccion arreglo[5]=6487556 valor arreglo[5]=0
direccion arreglo[6]=6487560 valor arreglo[6]=7017552
direccion arreglo[7]=6487564 valor arreglo[7]=0
direccion arreglo[8]=6487568 valor arreglo[8]=7017520
direccion arreglo[9]=6487572 valor arreglo[9]=0

direccion=7017520 *valor=7035104
direccion=7017524 *valor=0
direccion=7017528 *valor=7012688
direccion=7017532 *valor=0
direccion=7017536 *valor=1702258012
direccion=7017540 *valor=1275082801
direccion=7017544 *valor=1414657160
direccion=7017548 *valor=201368830
direccion=7017552 *valor=7035104
direccion=7017556 *valor=0
```

b.- Modifica la asignación inicial de “*ptr” de tal modo que ahora se utilice la función calloc, compila, ejecuta y explica la diferencia con la ejecución previa.

```
7 | int *ptr = calloc(10, sizeof(int));
```

En la función calloc se utilizan dos parámetros, el primero es el número de localidades a reservar y el segundo el tamaño.

```

direccion arreglo[0]=-1188522176    valor arreglo[0]=35
direccion arreglo[1]=-1188522172    valor arreglo[1]=40
direccion arreglo[2]=-1188522168    valor arreglo[2]=45
direccion arreglo[3]=-1188522164    valor arreglo[3]=50
direccion arreglo[4]=-1188522160    valor arreglo[4]=55
direccion arreglo[5]=-1188522156    valor arreglo[5]=32759
direccion arreglo[6]=-1188522152    valor arreglo[6]=-1529085820
direccion arreglo[7]=-1188522148    valor arreglo[7]=-969909159
direccion arreglo[8]=-1188522144    valor arreglo[8]=-1188521488
direccion arreglo[9]=-1188522140    valor arreglo[9]=32759

direccion=41714192    *valor=0
direccion=41714196    *valor=0
direccion=41714200    *valor=0
direccion=41714204    *valor=0
direccion=41714208    *valor=0
direccion=41714212    *valor=0
direccion=41714216    *valor=0
direccion=41714220    *valor=0
direccion=41714224    *valor=0
direccion=41714228    *valor=0

```

Lo que sucede a diferencia que en el inciso anterior es que con `calloc` en el arreglo dinámico se imprime valor de 0, porque aunque no este inicializado el arreglo, primero limpia las localidades de memoria que toma `calloc()`, y por eso siempre que se utilice `calloc()` el valor sera 0, porque primero lo limpia.

c.- Agrega instrucciones para asignar valores en las localidades de memoria reservadas con memoria dinámica. (deberás asignar valores múltiplos de 3 (3,6,9...)) Se deberá mostrar en pantalla los valores asignados y la dirección de estos.

```

18     for(cont=0;cont<10;cont++)
19     {
20         *(ptr+cont)=(cont+1)*3;
21         printf("direccion=%d    *valor=%d \n",ptr+cont, *(ptr+cont));
22     }
23     printf("\n\n");

```

En la línea 20 `*(ptr+cont)` lo igualamos a `cont +1` y su valor será multiplicado por 3. Coloco `cont +1` porque el arreglo empieza en cero, pero la multiplicación debe empezar en 1.

d.- Agrega instrucciones para utilizar la función realloc para redimensionar “ptr” con un nuevo tamaño de 25 espacios

- Utiliza realloc para asignar el nuevo tamaño en el mismo apuntador ptr

```
16 ptr=(int*)realloc(ptr,25*sizeof(int));
```

Se utiliza la función realloc y como parámetro se usa el apuntador a modificar y el nuevo tamaño. Para imprimirlo al ciclo for se cambió el valor 10 por el valor 20

```
19 for(cont=0;cont<10;cont++)
```

```
19 for(cont=0;cont<25;cont++)
```

- Utiliza realloc para asignar el nuevo tamaño en el apuntador ptr3

```
7 int *ptr3;  
8 int *ptr = calloc(10,sizeof(int));  
9 char *ptr2 = malloc(100*sizeof(char));  
10 ptr3 = realloc(ptr,25*sizeof(int));  
11 //Imprimiendo direcciones de un array
```

En la línea 7, primero se declara el apuntador ptr3 y luego en la línea 10 se asocia a la función realloc para asignar el nuevo tamaño. Se utiliza ptr ya que es el apuntador al cual se le redimensionara su tamaño.

Imprime las direcciones y los valores del apuntador obtenido con realloc
Indica si se conservan los valores que se habían asignado previamente

Realloc en el apuntador ptr

direccion=60784752	*valor=3
direccion=60784756	*valor=6
direccion=60784760	*valor=9
direccion=60784764	*valor=12
direccion=60784768	*valor=15
direccion=60784772	*valor=18
direccion=60784776	*valor=21
direccion=60784780	*valor=24
direccion=60784784	*valor=27
direccion=60784788	*valor=30
direccion=60784792	*valor=33
direccion=60784796	*valor=36
direccion=60784800	*valor=39
direccion=60784804	*valor=42
direccion=60784808	*valor=45
direccion=60784812	*valor=48
direccion=60784816	*valor=51
direccion=60784820	*valor=54
direccion=60784824	*valor=57
direccion=60784828	*valor=60
direccion=60784832	*valor=63
direccion=60784836	*valor=66
direccion=60784840	*valor=69
direccion=60784844	*valor=72

Realloc en el apuntador ptr3

direccion=7541808	*valor=3
direccion=7541812	*valor=6
direccion=7541816	*valor=9
direccion=7541820	*valor=12
direccion=7541824	*valor=15
direccion=7541828	*valor=18
direccion=7541832	*valor=21
direccion=7541836	*valor=24
direccion=7541840	*valor=27
direccion=7541844	*valor=30
direccion=7541848	*valor=33
direccion=7541852	*valor=36
direccion=7541856	*valor=39
direccion=7541860	*valor=42
direccion=7541864	*valor=45
direccion=7541868	*valor=48
direccion=7541872	*valor=51
direccion=7541876	*valor=54
direccion=7541880	*valor=57
direccion=7541884	*valor=60
direccion=7541888	*valor=63
direccion=7541892	*valor=66
direccion=7541896	*valor=69
direccion=7541900	*valor=72
direccion=7541904	*valor=75

Los valores si se conservan, las direcciones de memoria son diferentes

Finalmente indica si hay diferencias entre asignar realloc al mismo apuntador (ptr) o a uno nuevo (ptr3)

La diferencia es que al asignarla al mismo apuntador la dirección de memoria es la misma, sin embargo, al asignarla al apuntador diferente la memoria cambia

En general lo que hace este programa es imprimir las direcciones de memoria y los valores de un arreglo estático y después imprimir las direcciones de memoria y los valores de un arreglo dinámico, en el cual se prueban lo que hacen las funciones malloc(), calloc() y realloc() y lo que imprimen, finalmente con free liberan la memoria de los apuntadores. Para esto se necesitan conocimientos de como utilizar y la sintaxis de las funciones malloc, calloc, realloc, sizeof y realloc

En este ejercicio se logro un 100%, se cumplieron todos los requisitos de cada inciso que se pedía; se compilo y ejecuto correctamente.

Para poder realizar este programa se necesitan de elementos teóricos como arreglos, como se usan y para que son las funciones malloc, calloc y realloc ya que estas se utilizan al asignar memoria dinámica, asignan memoria para arreglos.

```
6 | int variable;  
7 | int *ptr = calloc(10,sizeof(int));  
8 | char *ptr2 = malloc(100*sizeof(char));  
9 | int *prt3 = realloc(ptr,25*sizeof(int));  
10 | //-----
```

También se necesitan conocimientos de como acceder y manipular un arreglo de memoria dinámica, ya que esto se necesita para poder acceder al arreglo y así imprimir su dirección

```
for(cont=0;cont<25;cont++)  
{  
    *(ptr+cont)=(cont+1)*3;  
    printf("direccion=%d   *valor=%d \n",ptr+cont, *(ptr+cont));  
}  
printf("\n");
```

Para resolver este problema lo primero que hice fue compilar y ejecutar el archivo que el profesor nos proporcionó, después analice el código con la salida y verificar el porqué de la salida, después agregue las instrucciones que pedían, donde tuve problema es en imprimir múltiplos de 3 ya que no me imprimía porque el primer valor a multiplicar fue 0, entonces verifique mis apuntes y lo logre.

Ejercicio 2

Codifica, compila y ejecuta el código "ejercicio2" posteriormente responde las preguntas.

a.- Explica los resultados que se muestran en pantalla.

Primero se imprime el tamaño de la estructura Alumno el cual es de 88 bytes, después se muestran las direcciones de memoria asignadas para Alumno, se utiliza malloc, cada dirección de memoria corresponde a un elemento del arreglo din1. Después hay un segundo apuntador, din2, se muestran las direcciones de memoria asignadas para la estructura Alumno, cada dirección de memoria corresponde a un elemento del arreglo din2. Las direcciones de memoria se imprimen con un ciclo for en cada una.

```

din1 = malloc(5*sizeof(Alumno));
din2 = calloc(5,sizeof(Alumno));

printf("Primer apuntador: \n");
for(j=0;j<5;j++){
    printf("Direccion[%d]=%d \n",j,din1+j);
}
printf("\n");

printf("Segundo apuntador \n");
for(j=0;j<5;j++){
    printf("Direccion[%d]=%d \n",j,din2+j);
}
```

Finalmente, con realloc se aumenta el tamaño de 10 elementos del apuntador 2, finalmente se imprimen las direcciones de memoria

```

printf("\n Con realloc: \n");
din3 = realloc(din2,10);
for(j=0;j<10;j++){
    printf("&din3[%d]=%d \n",j,din3+j);
}
```

```
C:\Users\eve18\Downloads\pr X +
Tamaño de objeto Alumno = 88
Primer apuntador:
Direccion[0]=11998256
Direccion[1]=11998344
Direccion[2]=11998432
Direccion[3]=11998520
Direccion[4]=11998608

Segundo apuntador
Direccion[0]=11998704
Direccion[1]=11998792
Direccion[2]=11998880
Direccion[3]=11998968
Direccion[4]=11999056

Con realloc:
&din3[0]=11998704
&din3[1]=11998792
&din3[2]=11998880
&din3[3]=11998968
&din3[4]=11999056
&din3[5]=11999144
&din3[6]=11999232
&din3[7]=11999320
&din3[8]=11999408
&din3[9]=11999496
```

b.- ¿Cuál es el tamaño de la estructura Alumno? ¿Cómo se obtiene ese tamaño?

El tamaño de la estructura es de 88 bytes y se obtiene con la función sizeof.

```
typedef struct {
    int numCuenta;
    char nombre[15];
    char apellido[15];
    float promedio;
    Direccion domicilio;
}Alumno;
```

Alumno	
int numCuenta	= 4 bytes
char nombre	= 15 bytes
char apellido	= 15 bytes
float promedio	= 4 bytes
Total	= 38 bytes


```

3  typedef struct {
4      char calle[20];
5      int num;
6      char colonia[20];
7      int cPostal;
8  }Direccion;

```

Dirección

char calle = 20 bytes
 int num = 4 bytes
 char colonia = 20 bytes
 int cPostal = 4 bytes
 Total = 48 bytes

38 bytes + 48 bytes = 86 bytes

¿Por qué se imprime 88 bytes?

Esto sucede porque el compilador como se alinea la memoria hay un relleno que se realiza para así garantizar un acceso más eficiente en los datos. La memoria es múltiplo de 4, por lo que el relleno en este caso es en los arreglo char [15], a pesar de que valen 15 bytes, en memoria ocupan 16 porque es múltiplo de 4, entonces con la explicación en lo anterior queda así.

```

typedef struct {
    int numCuenta;
    char nombre[15];
    char apellido[15];
    float promedio;
    Direccion domicilio;
}Alumno;

```

Alumno

int numCuenta = 4 bytes
 char nombre = 16 bytes
 char apellido = 16 bytes
 float promedio = 4 bytes
 Total = 40 bytes

```

3  typedef struct {
4      char calle[20];
5      int num;
6      char colonia[20];
7      int cPostal;
8  }Direccion;

```

Dirección

char calle = 20 bytes
 int num = 4 bytes
 char colonia = 20 bytes
 int cPostal = 4 bytes
 Total = 48 bytes

40 bytes + 48 bytes = 88 bytes

c.- Agrega las instrucciones al código para pedir al usuario los datos de los alumnos que se reservan en el programa

Para esto agrego 3 funciones:

La primera es obtenerAlumno, en la cual se pide al usuario los datos y los obtiene con scanf

```
void obtenerAlumno(Alumno *alumno)
{
    printf("Numero de cuenta: ");
    scanf("%d", &alumno->numCuenta);
    printf("Nombre: ");
    scanf("%s", alumno->nombre);
    fflush(stdin);
    printf("Apellido: ");
    scanf("%s", alumno->apellido);
    fflush(stdin);
    printf("Promedio: ");
    scanf("%f", &alumno->promedio);
}
```

La segunda es crearDireccion, de tipo Direccion tiene como parámetros los elementos de la estructura Direccion. En esta función lo primero que se hace es crear una variable de tipo Alumno, después con strcpy copia cada parámetro dentro de los campos correspondientes de la estructura Alumno, finalmente devuelve la variable creada.

```
--
56 Direccion crearDireccion(char *calle, int numero, char* col, int postal)
57 {
58     Direccion direccionCreada;
59     strcpy(direccionCreada.calle, calle);
60     direccionCreada.num = numero;
61     strcpy(direccionCreada.colonia, col);
62     direccionCreada.cPostal = postal;
63     return direccionCreada;
64 }
```

La tercera función es agregarDireccion, toma como parámetro un apuntador de tipo Alumno, aquí se piden al usuario los datos (elementos de la estructura) y los obtiene con scanf

```

66 void agregarDireccion(Alumno *alumno)
67 {
68     printf("Calle: ");
69     scanf("%s", alumno->domicilio.calle);
70     fflush(stdin);
71     printf("Numero: ");
72     scanf("%d", &alumno->domicilio.num);
73     printf("Colonia: ");
74     scanf("%s", alumno->domicilio.colonia);
75     fflush(stdin);
76     printf("Codigo postal: ");
77     scanf("%d", &alumno->domicilio.cPostal);
78 }

```

Después en el main los mando a llamar:

```

obtenerAlumno(&din1[j]);
agregarDireccion(&din1[j]);    imprimirAlumno(din1[j]);

```

```

ALUMNOS
Alumno 1
Numero de cuenta: 321184191
Nombre: Evelin
apellido: Jimenez
Promedio: 9.8
Calle: Misisipi
Numero: 18
Colonia: Acuitlapilco
Codigo postal: 44373
Alumno 2
Numero de cuenta: 321352923
Nombre: Patricia
apellido: Gonzales
Promedio: 7.4
Calle: Balsas
Numero: 19
Colonia: Acuitlapilco
Codigo postal: 43232
Alumno 1
Numero de cuenta: 321184191
Name: Evelin
LastName : Jimenez
Calle: Misisipi
Numero de tu direccion: 18
Colonia: Acuitlapilco
Codigo Postal: 44373

```

```

Alumno 2
Numero de cuenta: 321352923
Name: Patricia
LastName : Gonzales
Calle: Balsas
Numero de tu direccion: 19
Colonia: Acuitlapilco
Codigo Postal: 43232

```

d.- Explica de qué forma se podría liberar la memoria reservada por la función calloc o malloc sin utilizar la función free, realiza el intento en código e indica tus resultados

Para poder liberar la memoria utilizamos la función realloc para liberar la memoria asignada previamente con malloc, calloc o realloc, pasando como argumento el apuntador a limpiar y como tamaño cero, después establezco el apuntador a NULL, prosigo a verificar que si el apuntador es igual a NULL la memoria se ha liberado correctamente, si no se manda el mensaje de que la memoria no se liberó.

```
din1=realloc(din1,0);
din1=NULL;
if(din1==NULL)
{
    printf("La memoria se ha liberado\n");
}
else
{
    printf("Error: la memoria no se ha liberado");
}
din3 = realloc(din2,0);
din3=NULL;
if(din3==NULL)
{
    printf("La memoria se ha liberado\n");
}
else
{
    printf("Error: la memoria no se ha liberado");
}
//free(din1); //
```

Aquí los apuntadores previamente asignados dinámicamente con malloc y calloc, utilizamos realloc para asignarles un tamaño de 0 y después las apuntamos a NULL, después se verifica si los apuntadores si son igual a NULL la memoria se libero correctamente

```
La memoria se ha liberado
La memoria se ha liberado
Presione una tecla para continuar
```

Hay 2 archivos un archivo.h llamado "Alumno.h" y un archivo.c llamado "ejercicio2.c".

En el archivo "Alumno.h" están definidas las estructuras Direccion con sus elementos (calle, num, colonia y cPostal) y la estructura Alumno (numCuenta, nombre, apellido y promedio) y adentro la estructura Direccion.

Después se declaran las funciones y se definen cada una.

En el archivo .c esta la función principal, dentro de ella se obtiene el tamaño de la estructura Alumno, usando la función sizeof.

```
3 | int j;  
4 | printf("Tamaño de objeto Alumno = %d \n", sizeof(Alumno));
```

Luego con ciclos for se imprimen las direcciones de 2 apuntadores, que previamente se les asigno el tamaño con las funciones malloc y calloc, respectivamente, igual con ciclo for se manda a llamar las funciones “obtenerAlumno”, “agregarDireccion” e “imprimirAlumno” para que así se piden los datos de la estructura Alumno y se imprima la información que ingresa el usuario, luego con realloc se redimensiona el tamaño del segundo apuntador y se imprimen las direcciones e igual se mandan a llamar las funciones, después se libera memoria sin utilizar free

```
.....  
din1 = malloc(5*sizeof(Alumno));  
din2 = calloc(5, sizeof(Alumno));  
  
printf("Primer apuntador: \n");  
for(j=0; j<5; j++)  
{  
    printf("Direccion[%d]=%d \n", j, din1+j);  
}  
  
printf("ALUMNOS\n");  
for(j=0; j<2; j++)  
{  
    printf("Alumno %d \n", j+1);  
    obtenerAlumno(&din1[j]);  
    agregarDireccion(&din1[j]);  
}  
for(j=0; j<2; j++)  
{  
    printf("Alumno %d\n", j+1);  
    imprimirAlumno(din1[j]);  
}
```

Este programa cumple con el 100% de requisitos de cada inciso y las explicaciones que requiere.

Para realizar este programa se necesitan de conocimientos de estructuras, porque en el programa se utilizan las estructuras Direccion y Alumno.

También se necesitan conocimientos de funciones malloc, calloc, realloc y sus argumentos, las funciones sizeof para obtener el tamaño de la estructura Alumno y se utilizan las funciones free en algún momento para liberar memoria.

```

4      printf(" tamaño de objeto Alumno = %d\n", sizeof(Alumno));
5      Alumno *din1,*din2,*din3;
6      din1 = malloc(5*sizeof(Alumno));
7      din2 = calloc(5,sizeof(Alumno));
8

```

Se utiliza realloc para liberar la memoria.

```

52      din1=realloc(din1,0);
53      din1=NULL;
54

```

También se necesitan conocimientos de funciones en general para poder optimizar el programa.

Igual se necesitan conocimientos de cómo crear nuestras propias bibliotecas para que en un archivo.h estén las estructuras y las funciones y en otro archivo.c este la función principal y relacionarlas

Resolví este problema de una manera que primero compile y ejecute el ejercicio2.c que nos proporciono el profesor para así analizar la salida, posteriormente analice el tamaño de la estructura que nos indicaba la salida del programa y el porque de ese tamaño, pero sumando los elementos me daba diferente al que se indicaba la salida por lo que investigue en distintas fuentes y así encontré la explicación del porque daba más.

Las fuentes son:

https://cv.uoc.edu/annotation/8255a8c320f60c2bfd6c9f2ce11b2e7f/619469/PID_00218275/PID_00218275.html

<https://www.fing.edu.uy/tecnoinf/mvd/cursos/arqcomp/material/teo/arq-teo09.pdf>

Después me base en ejercicios anteriores para poder ingresar las instrucciones para pedir los datos reservados en la memoria.

Finalmente, para poder liberar memoria sin utilizar free solo pensé en que si realloc redimensiona el tamaño entonces poniéndole 0 como argumento y un NULL se liberaba la memoria

Ejercicio 3

✓ Diseña un tipo abstracto de dato llamado Automovil en el cual puedas encapsular elementos

marca, modelo, placas, etc.

✓ Elabora un programa en el cual se incluya la definición de tu tipo abstracto de dato, y utilizando memoria dinámica crea un “arreglo dinámico de automóviles” en el cual se solicite al usuario en tiempo de ejecución, el tamaño de dicho arreglo.

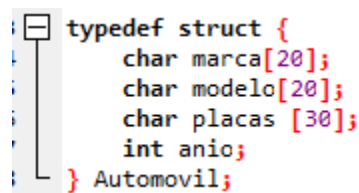
✓ Dentro de la función principal del programa, el usuario deberá indicar y llenar el número de autos acorde con el tamaño del arreglo.

✓ Recuerda que es recomendable el uso de funciones para trabajar con implementaciones de tipo abstracto de datos.

✓ Realiza las pruebas necesarias ejecutando tu código y elabora tus conclusiones

Lo que hace el programa es pedir un tamaño de un arreglo dinámico de automóviles, después se piden ingresar los datos del arreglo conforme al tamaño que el usuario indico, posteriormente se imprimen esos datos de cada Automóvil.

Para esto se crea un archivo.h, en el cual se declara la estructura “Automóvil”, que tiene como elementos: marca, modelo, placas y año.

A screenshot of a code editor with a light blue background. On the left, there is a vertical toolbar with icons for file operations. The code is written in a C-like syntax, using 'typedef struct' to define a type named 'Automovil'. The struct contains four members: 'marca' (char array of size 20), 'modelo' (char array of size 20), 'placas' (char array of size 30), and 'anio' (int). The closing brace of the struct is followed by the name 'Automovil'.

Después se declaran tres funciones:

La primera que es de tipo Automóvil que se llama “crearAutomovil” que tiene como parámetros los elementos de la estructura. En esta función lo primero que se hace es crear una variable de tipo Automóvil, después con strcpy copia cada parámetro dentro de los campos correspondientes de la estructura Automóvil, finalmente devuelve la variable creada.

```
Automovil crearAutomovil(char *mar, char *model, char *placas, int anio)
{
    Automovil automovilCreado;
    strcpy(automovilCreado.marca,mar);
    strcpy(automovilCreado.modelo,model);
    strcpy(automovilCreado.placas,placas);
    automovilCreado.anio = anio;
    return automovilCreado;
}
```

La segunda función es obtenerAutomovil, esta recibe como argumento un apuntador a la estructura Automóvil. Solicita al usuario que se ingrese información sobre el automóvil (marca, el modelo, las placas y el año), se leen con scanf, se utiliza fflush, para limpiar el buffer de entrada

```
void obtenerAutomovil(Automovil *automovil)
{
    printf("Marca: ");
    scanf("%s",automovil->marca);
    fflush(stdin);
}
```

La tercera función es imprimirAutomovil, aquí se imprime cada elemento del automóvil

```
37 void imprimirAutomovil(Automovil automovilPrint)
38 {
39
40     printf("Marca: %s \n",automovilPrint.marca);
}
```

Luego esta un archivo .c llamado principal el cual contiene la función principal main, en esta función se pide al usuario que ingrese el tamaño del arreglo, posteriormente se guarda en una variable llamada "tamanio", después se declara un apuntador de tipo Automóvil, después a ese apuntador se le asigna la memoria dinámica que el usuario ingreso, para esto se utiliza malloc.

```
5     printf("Escribe de que tamaño quieres tu arreglo: ");
6     scanf("%d",&tamanio);
7
8     Automovil *din1;
9     din1 = malloc(tamanio*sizeof(Automovil));
```

Luego con un ciclo for se pide al usuario que ingrese la información de cada automóvil, para esto se manda a llamar la función obtenerAutomovil, iterando sobre la dirección de cada elemento del arreglo.

Después para obtener los datos, se imprimen para esto igual se utiliza un ciclo for que imprime la información de cada Automóvil.

Finalmente se utiliza la función free para liberar la memoria.


```

10 for(j=0;j<tamano;j++)
11 {
12     printf("Ingresa los datos del automovil %d\n",j+1);
13     obtenerAutomovil(&din1[j]);
14 }
15 for(j=0;j<tamano;j++)
16 {
17     printf("Automovil: %d\n",j+1);

```

```

C:\Users\eve18\OneDrive\Doc >
Escribe de que tamaño quieres tu arreglo: 2
Ingresa los datos del automovil 1
Marca: Honda
Modelo: Civil
Placas: LVBXB250
Año: 2014
Ingresa los datos del automovil 2
Marca: Mazda
Modelo: Sport
Placas: PR05MDB
Año: 2024
Automovil: 1
Marca: Honda
Modelo: Civil
Placas: LVBXB250
Año: 2014
Automovil: 2
Marca: Mazda
Modelo: Sport
Placas: PR05MDB
Año: 2024

-----
Process exited after 63.28 seconds with return value 0
Presione una tecla para continuar . . . |

```

El programa cumple el 100% de los requisitos que se pide, se pide el arreglo y se llenan los elementos de acuerdo con el tamaño que el usuario ingresa, posteriormente se imprime.

Para poder realizar este programa se necesitan conocimientos de funciones como malloc, free y sizeof, ya que para asignar el tamaño del arreglo dinámicamente se utiliza la función malloc tomando como argumento la variable "tamaño"

multiplicado por sizeof (Automóvil), igual se ocupan conocimientos de estructuras, ya que aquí se ocupa la estructura Automóvil.

```
9 | din1 = malloc(tamano*sizeof(Automovil));
```

También se ocupan conocimientos de funciones ya que para optimizar el programa se utilizan funciones como la función “crearAutomovil”, “obtenerAutomovil” e “imprimirAutomovil”, para después mandarlas a llamar.

```
10 Automovil crearAutomovil(char*, char*, char*, int);  
11 void obtenerAutomovil(Automovil*);  
12 void imprimirAutomovil(Automovil);  
13
```

Igual se necesitan conocimientos de como crear nuestras propias bibliotecas para que en un archivo.h estén las estructuras y las funciones y en otro archivo.c este la función principal y relacionarlas

Automovil.h	Principal.c
1 #include "Automovil.h"	
2 int main()	
3 {	

Para poder realizar el ejercicio, tome como referencia el ejercicio 2 de esta práctica, solo modifique la estructura y los elementos al igual que elimine algunas funciones.

Conclusiones

Si se cumplen los objetivos porque se utilizan las funciones que permiten reservar y almacenar información de manera dinámica, es decir se utilizan las funciones malloc, calloc y realloc en todos los ejercicios para reservar memoria dinámica. Igual se entiende el uso del apuntador en este caso aplicado en la aplicación de memoria dinámica.

Los ejercicios si contribuyeron al aprendizaje porque tenía dificultades de como se usaba la función realloc y como los apuntadores se relacionaban con las funciones así que analizando los ejercicios lo entendí.

Las posibles ventajas es que con las funciones los programas se optimizan, las posibles desventajas es que se usan estructuras y apuntadores el cual puede dificultar el entendimiento de los ejercicios ¿, si los conceptos no quedaron claros.

Critica constructiva es que las instrucciones de los ejercicios sean más claras, que sean como las instrucciones del ejercicio 3, instrucción por instrucción. Esto debido en que las instrucciones del ejercicio 2 no eran del todo claras.

El porcentaje global de todos los ejercicios son del 100% todos se cumplieron con los requisitos indicados.